2024

AMAP – Product management

TECHNICAL REPORT – SPRINT 1 – VERSÃO 1.0

[1090470 – BRUNO SANTOS]
[1190640 – GUILHERME COTA]
[1230213 – TOMÁS OLIVEIRA]
[1230196 – LUÍS SERAPICOS]
[1231304 – TELO GASPAR]

1 Introduction

AMAP (Associação pela Manutenção da Agricultura de Proximidade) or CSA (CSA – Comunidade que Sustenta a Agricultura) play an essential role in connecting producers and consumers of organic products, promoting sustainable practices and healthier eating.

This report presents the start of the application development process, which is based on the requirements gathering carried out previously and documented in the SRS (Software Requirements Specification). The current phase is dedicated to implementation, where the identified requirements are translated into concrete functionalities, taking into account the operational particularities of AMAPs and good software engineering practices.

This document details the technical progress made, as well as the strategies and tools adopted, providing a clear overview of the work carried out to date and the next steps towards completing the project.

1.1 Purpose

The purpose of this project is to develop an intuitive and efficient application that supports the management of AMAPs, promoting sustainability and the local economy. The solution aims to create a digital platform that facilitates the registration of producers and co-producers, the management of organic products and their involvement.

By offering features such as order organization and transparency in transactions, the application aims to help responsible consumption and access to quality organic food. It will also simplify current traditional processes, allowing for greater efficiency and operational simplification.

1.2 Methodology and Story mapping

An agile methodology was used to develop the application, using Scrum as the project management framework. This methodology was chosen for its flexibility and ability to adapt to change, which are fundamental characteristics for a project in its early stages.

Activities were organized into sprints, this being Sprint 1, with regular planning meetings, daily monitoring and a review at the end of each cycle. The Jira tool was used to manage the backlog, plan the sprints and monitor the progress of the tasks, providing a clear and collaborative view of the activities in progress.

Roles:

- Product Owner: Tomás Oliveira
- Scrum Master: Guilherme Cota
- Development Team: Bruno Santos, Tomás Oliveira, Guilherme Cota, Luís Serapicos, Telo Gaspar.

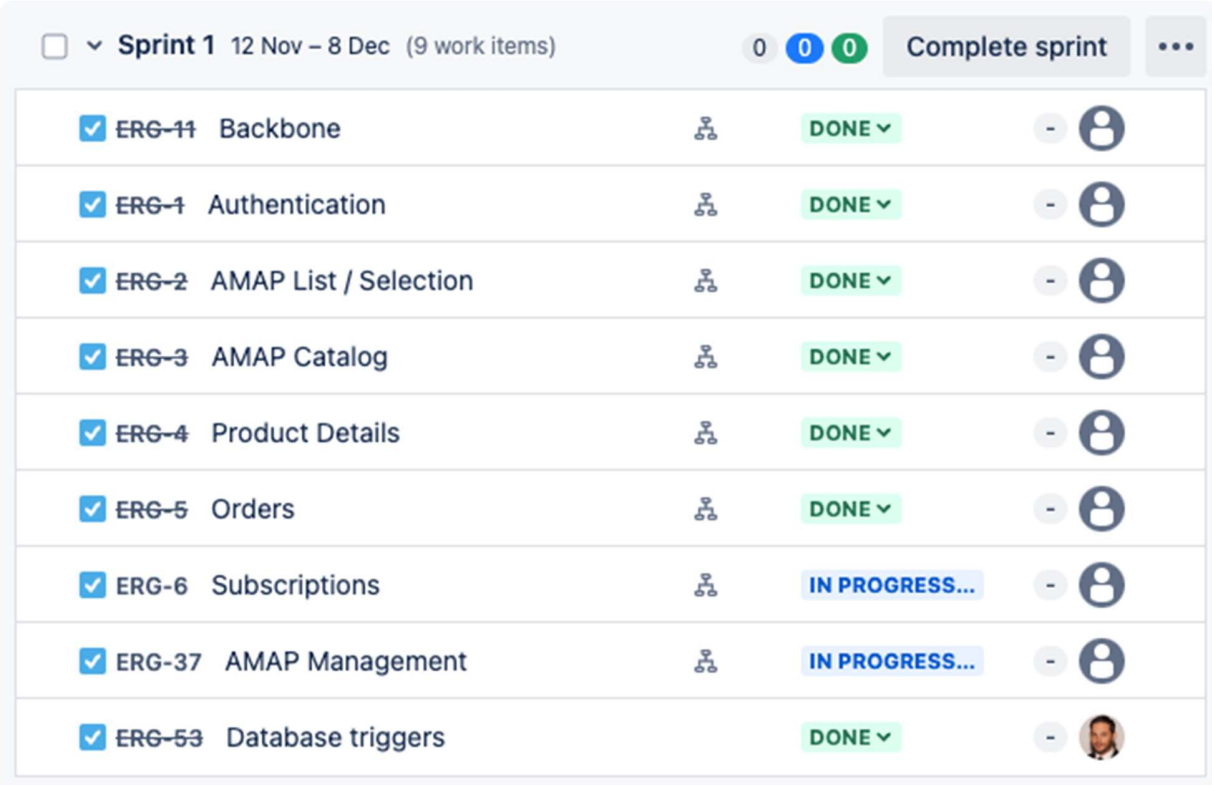


Figure 1 - Jira User stories

1.2.1 Backbone

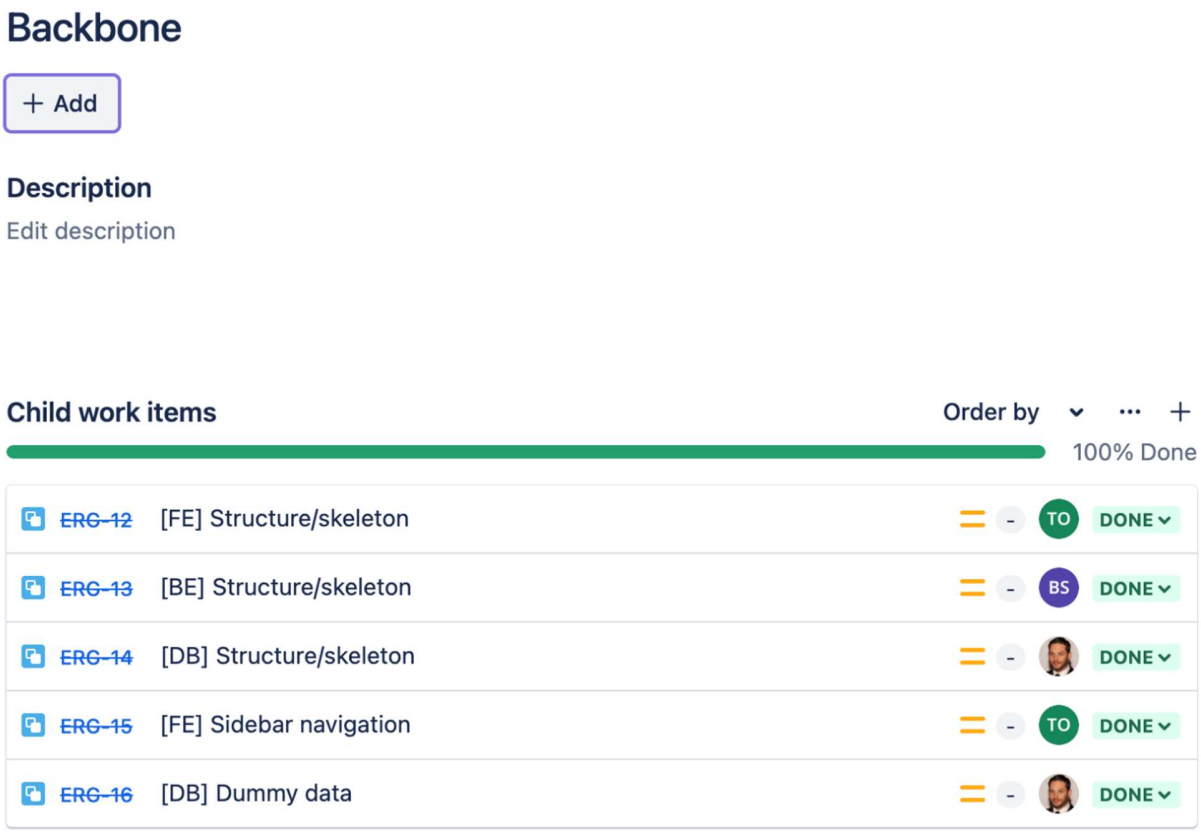


Figure 2 - Backbone tasks

1.2.2 Authentication

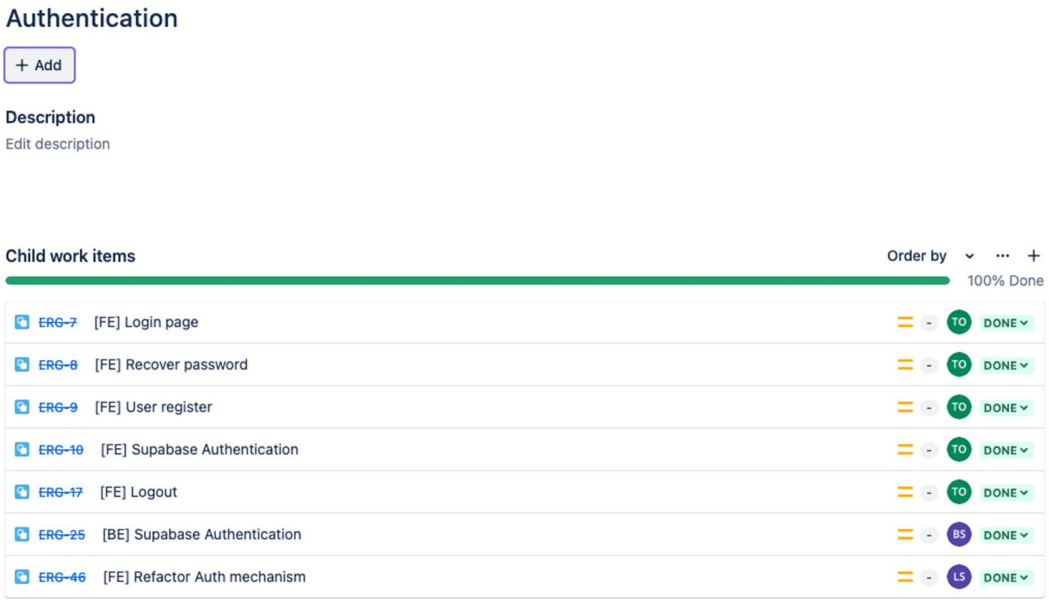


Figure 3 - Authentication tasks

1.2.3 AMAP List/Selection

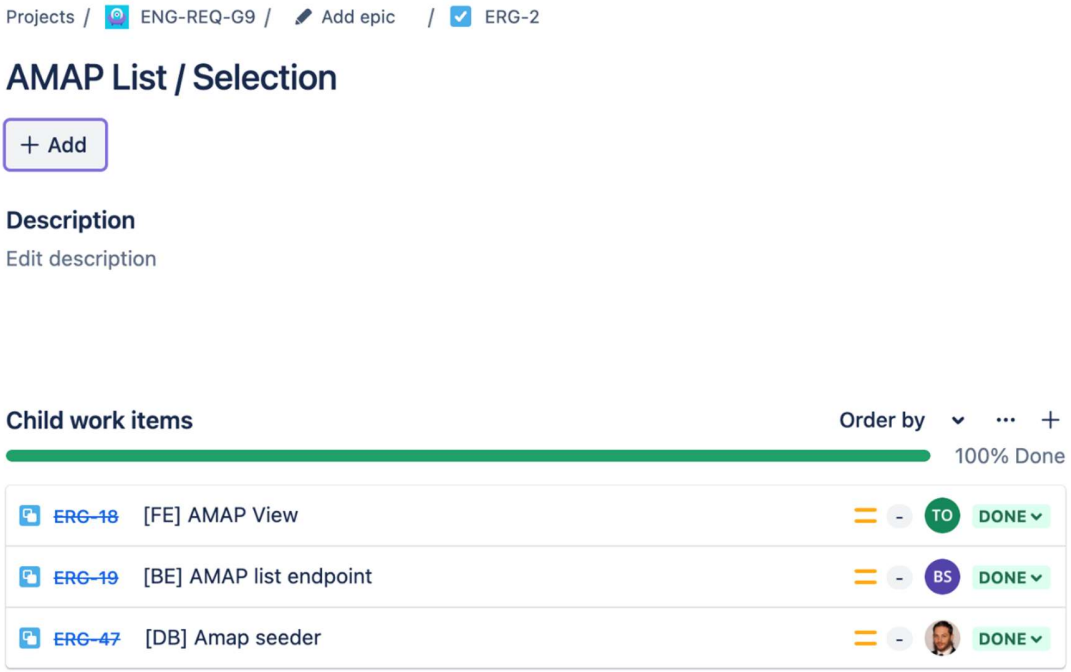


Figure 4 - AMAP list tasks

1.2.4 AMAP Catalog

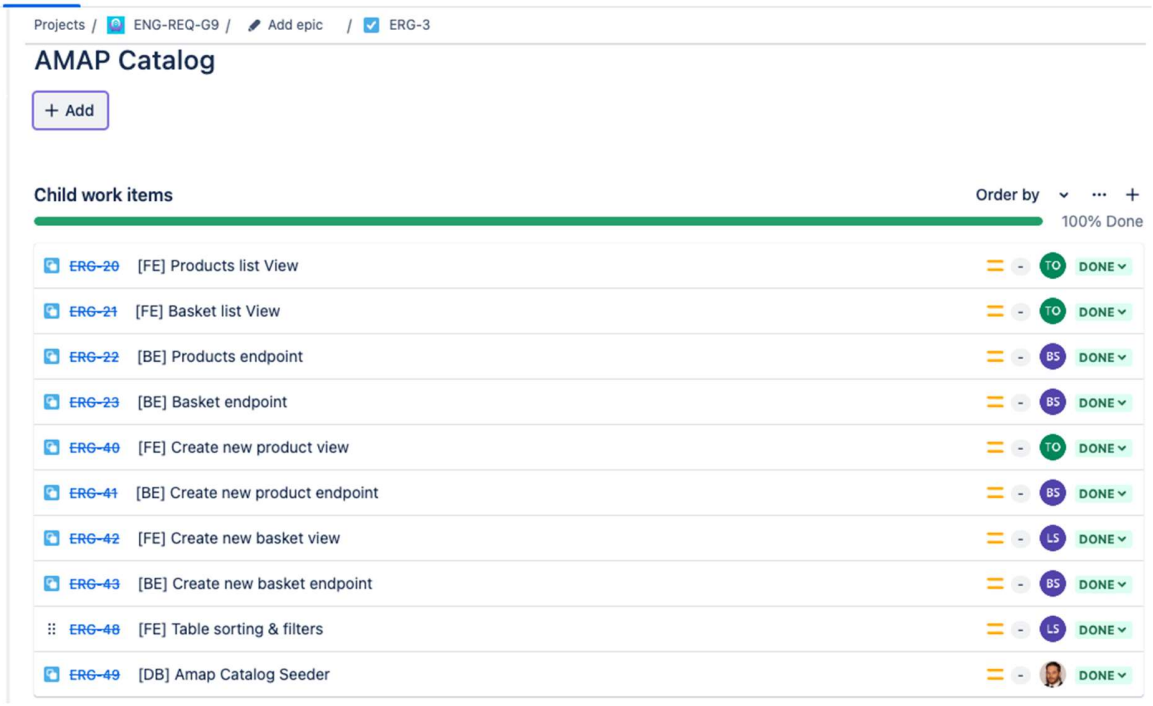


Figure 5 - AMAP catalog

1.2.5 Product Details

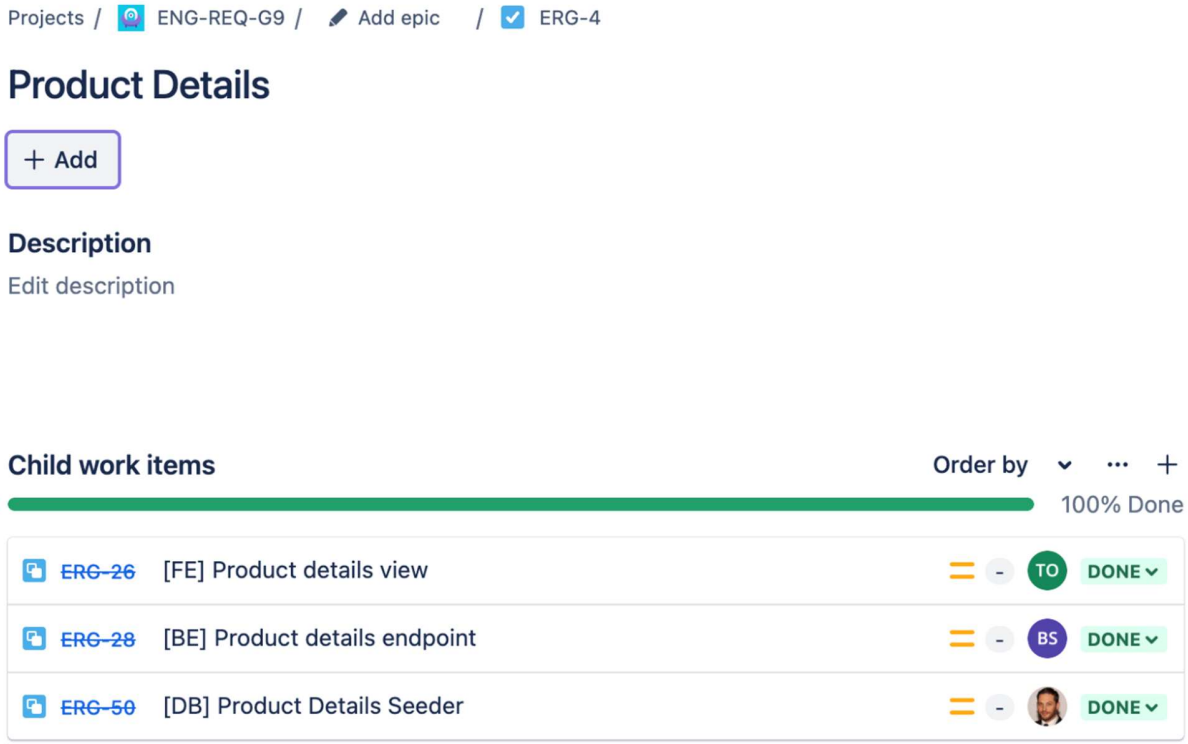


Figure 6 - Product details

1.2.6 Orders

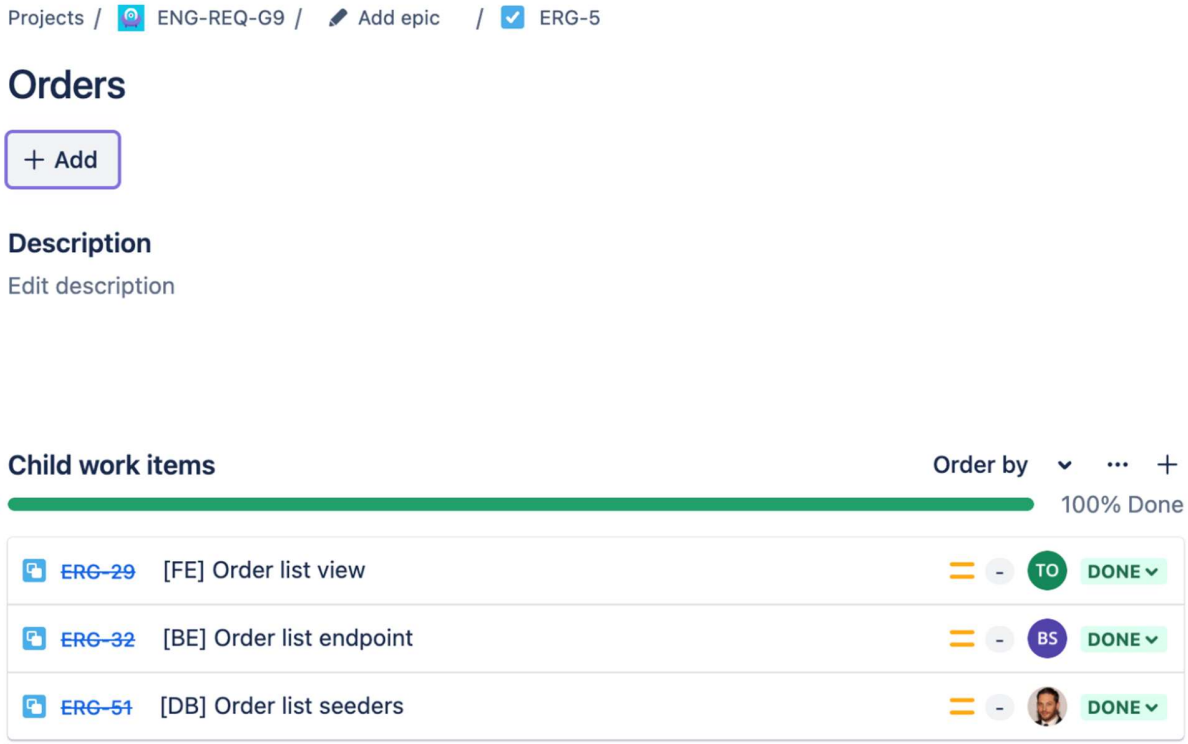


Figure 7 - Orders tasks

1.2.7 Subscriptions

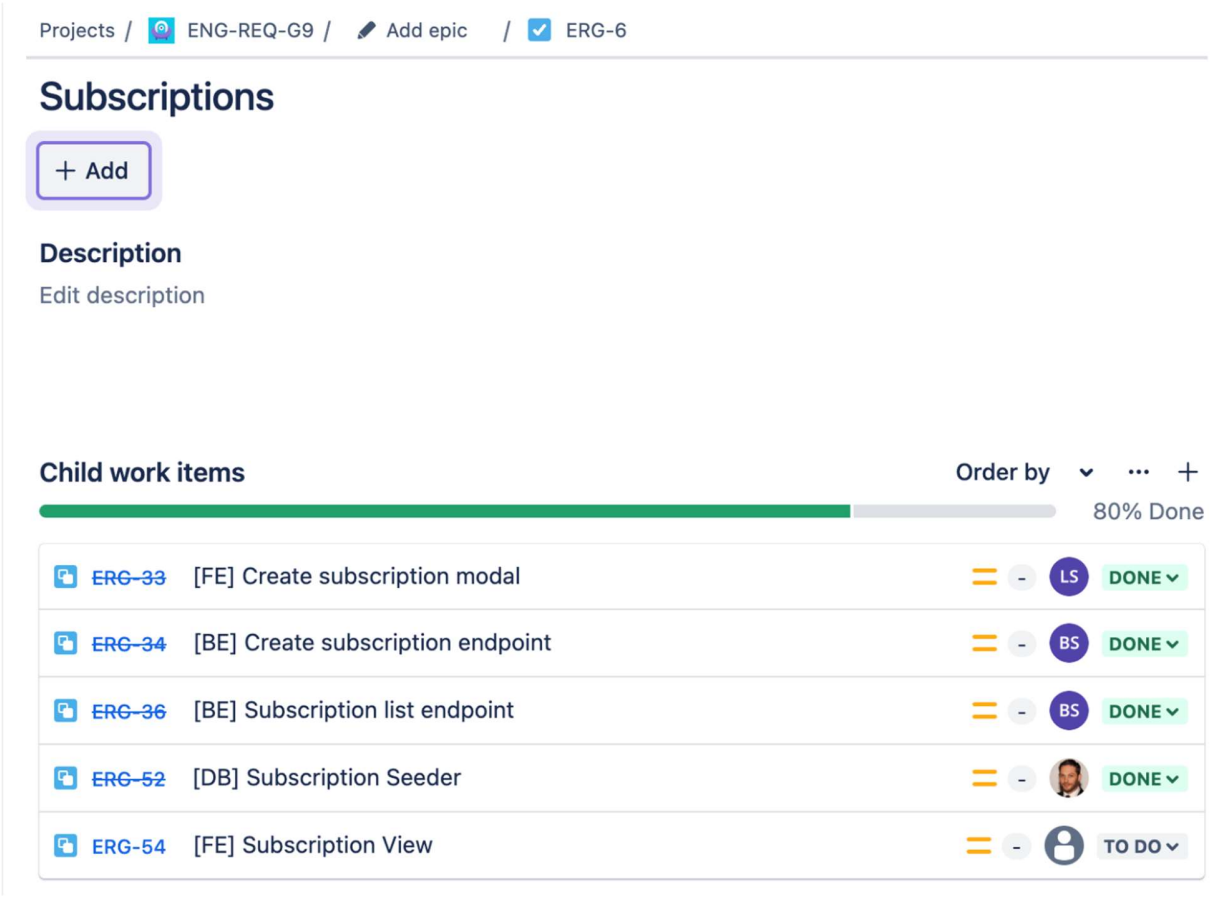


Figure 8 - Subscriptions tasks

1.2.8 AMAP Management

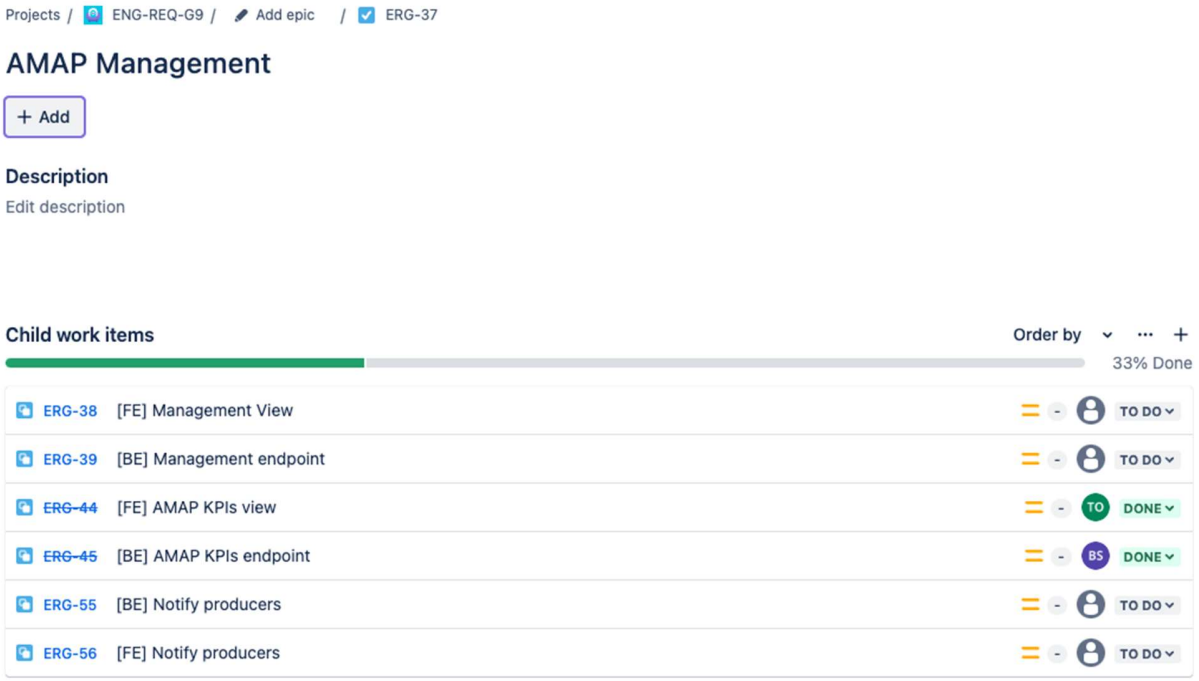


Figure 9 - AMAP management tasks

1.2.9 Database Triggers

Figure 10 - Database tasks

1.3 Meetings

As part of the application's development, sprint meetings played a central role in ensuring team coordination and compliance with the established objectives. These meetings were held regularly, following the Scrum methodology, as shown in Table X.

Table 1- Scrum meetings

Date	Meeting
12/11/2024	Sprint planning
19/11/2024	Daily meeting
26/11/2024	Daily meeting
28/11/2024	Daily meeting
03/12/2024	Daily meeting
04/12/2024	Daily meeting
05/12/2024	Daily meeting
07/12/2024	Daily meeting
08/12/2024	Sprint retrospective

1.4 Convention

To ensure consistency and organization during project development, clear conventions were established for naming, structuring and documenting the artifacts produced. The tasks managed in Jira were named in a standardized way, allowing easy identification of the type of activity and its relationship to the system requirements. In the code, the use of common practices in methods and variables was adopted. This choice ensured alignment with widely accepted standards in software engineering.

The structure of the code was planned to be modular, separating the interface, business logic and data persistence layers, which contributed to greater clarity and ease of development.

The project's source code was managed through a Git repository on GitHub. Only one repository was used, separating Frontend from Backend. The repository followed a clear organization of directories, and practices of frequent commits and detailed descriptions facilitated collaboration and organization within the team.

Figure 11 - Project repository

2 Technology

The application was developed using the latest technologies to allow scalability, performance and easy maintenance.

The backend used the Express framework for Node.js, which facilitates the use of a flexible and efficient API, guaranteeing simple and fast endpoint management.

React was used on the frontend to provide an interactive and responsive interface. React allows components to be created in a reusable way, facilitating code optimization and development, as well as scalability. The application was developed with a responsive design, so that it is compatible with desktop and mobile use, providing the user with a consistent experience across different platforms.

For authentication and data management, Supabase was integrated, an open-source platform that offers a robust solution for the database, authentication and APIs. Supabase was chosen for its simple integration with the front and backend, allowing users to be created, permissions to be controlled and interaction with the database to be secure and scalable.

For requirements management and functionality planning, the Miro tool was used for story mapping, which made it easier to organize and prioritize tasks. The code was managed using Git, on GitHub, which enabled efficient and organized collaboration during development.

3 System features

The application's requirements were surveyed and for each of them the tasks and their priority were identified.

In order to plan the work to be carried out, the strategy of division by Story Mappings was adopted. Following on from this chapter, all the tasks for preparing and controlling the work implemented are presented.

The backlog contains the tasks to be continued in the next sprint or to improve the solution in the future.

In terms of priorities, a criterion was followed to provide a solution to what was requested for this sprint, with priority being given to the structure of the solution and the database, in order to be able to implement the User stories.

Type	Key	Summary	Status	Sprint
	ERG-11	Backbone	TO DO	Sprint 1
	ERG-11 ERG-12	[FE] Structure/skeleton	TO DO	Sprint 1
	ERG-11 ERG-13	[BE] Structure/skeleton	TO DO	Sprint 1
	ERG-11 ERG-14	[DB] Structure/skeleton	TO DO	Sprint 1
	ERG-11 ERG-15	[FE] Side menu	TO DO	Sprint 1
	ERG-11 ERG-16	[DB] Dummy data	TO DO	Sprint 1

Figure 12 -Jira tasks

3.1 Component Diagram

The solution consists of three main components previously defined in the SRS. These are the Web Portal (frontend), the API (backend) for feeding the Web Portal, and the Supabase service for using the Database and Authentication. The web portal can be used on both desktop and mobile devices.

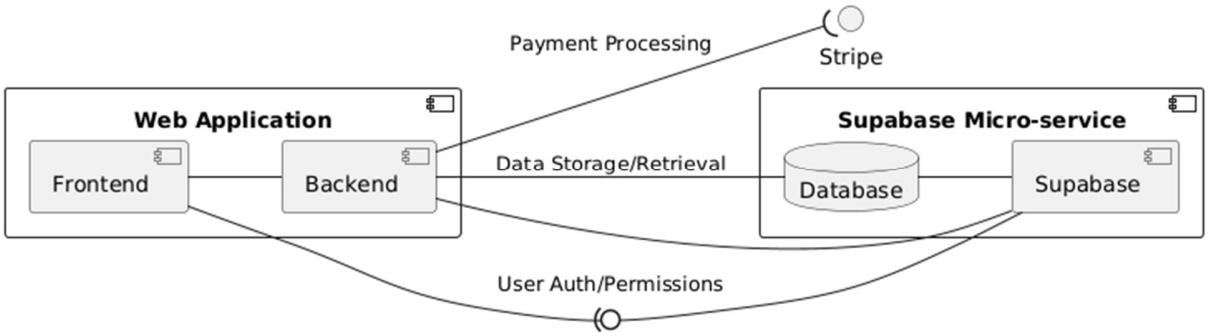


Figure 13 - Component diagram

3.2 Object Relational Model Diagram

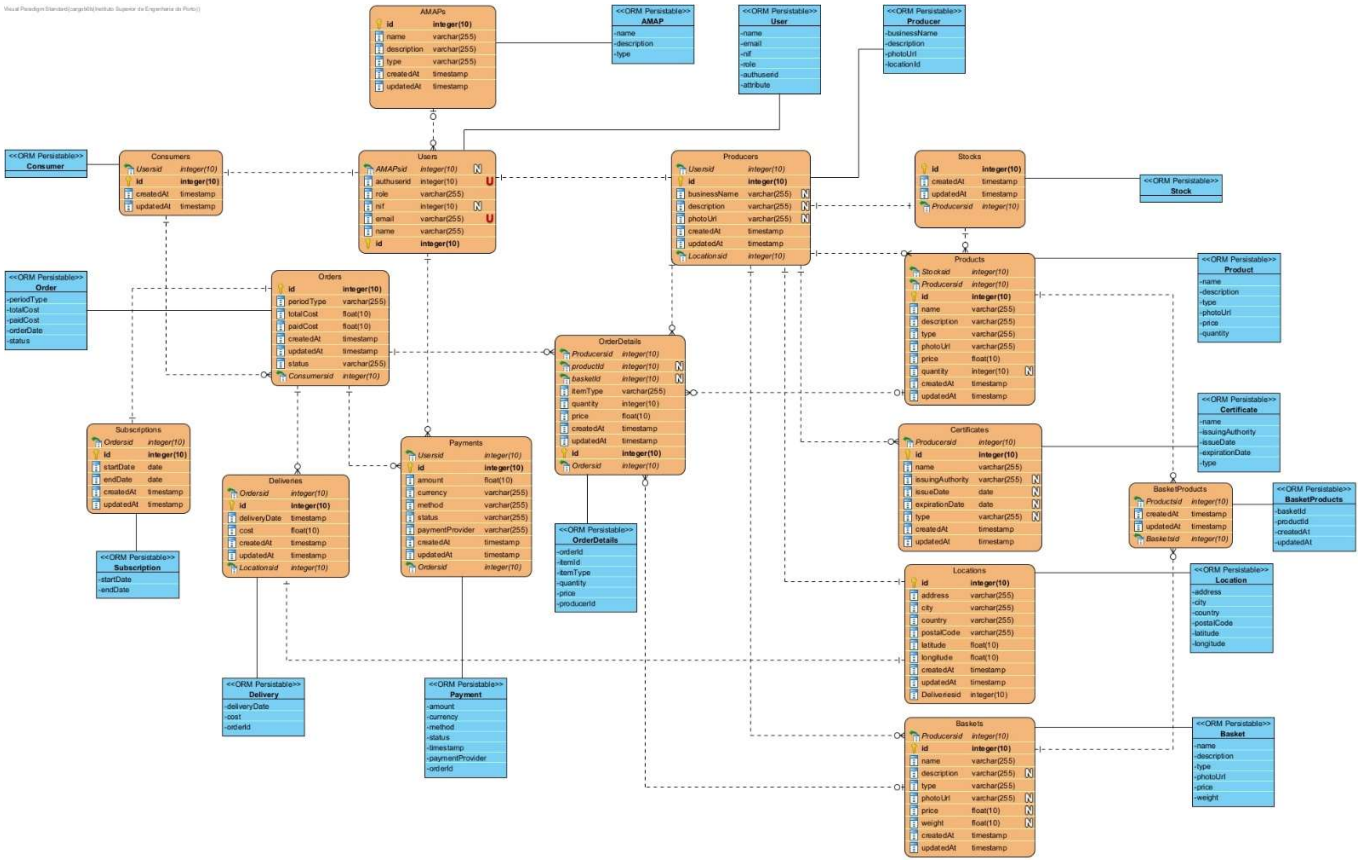


Figure 14 -Model diagram

The above diagram shows the entire object relational model of the project. Essentially is it mapping the ORM Persistable Classes (blue boxes) to Entities (orange boxes).

Entities specify tables in our database, each entity is specified with the necessary properties and relationships between tables.

ORM Persistable Classes are defined in the code of our project, they are classes that get mapped into a schema. Building the schema was done automatically through the use of an ORM like Sequelize. Some properties exist in the entities but not in the classes, however that is as intended, Sequelize doesn't require that we create all the exact fields, it does it for us.

4 Backbone

In order to develop the solution, it was necessary to create the entire project structure, which was then used to implement all the necessary user stories, which was divided as follows:

- Frontend structure
- Backend structure
- Database (tables and relations)

Figure 15 - Backbone tasks

4.1 Frontend

React was used to develop the solution on the frontend, and at this point it was necessary to create the entire structure of the web portal so that it could host all the functionalities. The structure was created according to the selected technology, but also following good practices.

All the necessary structure, middleware, communication services with the API were implemented. In order to communicate with the API, a Bearer token was implemented to ensure the security and authentication of the user's requests.

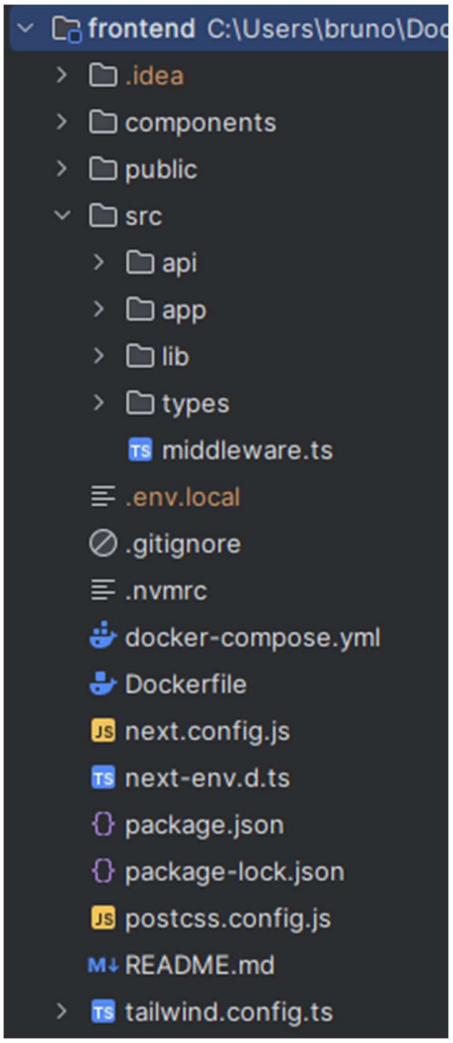


Figure 16 - FE structure

4.1.1 Middleware

We created a middleware using the @supabase/auth-helpers-nextjs library to manage authentication inside our Next.js application. The middleware first initializes a response object with NextResponse.next() to allow the request to proceed by default. A key aspect of its functionality is handling public and protected routes.

```
const res = NextResponse.next();

const publicUrls = ['/reset', '/login', '/register', '/confirm'];
```

It checks if the requested URL matches a predefined list of public paths, such as /reset, /login, /register, or /confirm.

If the request targets one of these public routes, it is allowed to continue without any further checks, ensuring users can access specific pages without authentication.

For all other routes, the middleware integrates with Supabase by creating a middleware client that links the request and response objects. This client is used to retrieve the current user session through the supabase.auth.getSession() method.

If no session is found, indicating that the user is not authenticated, the middleware redirects the user to the /login page. However, if a valid session exists, the request is processed as intended.

The middleware configuration includes a matcher that defines the routes it should handle. It excludes API endpoints, static assets, and other system files, ensuring these routes are unaffected by the authentication logic. This implementation effectively secures protected resources by allowing only authenticated users to access them while maintaining unrestricted access to public pages.

4.1.2 Public Pages

4.1.2.1 Login Page

A login page has been developed that utilizes Supabase for both authentication and data management within a Next.js application. This interface enables users to log in using their email and password, reset their passwords when necessary, and create new accounts. It features a responsive user interface that adjusts according to user interactions, providing distinct modes for logging in and recovering passwords.

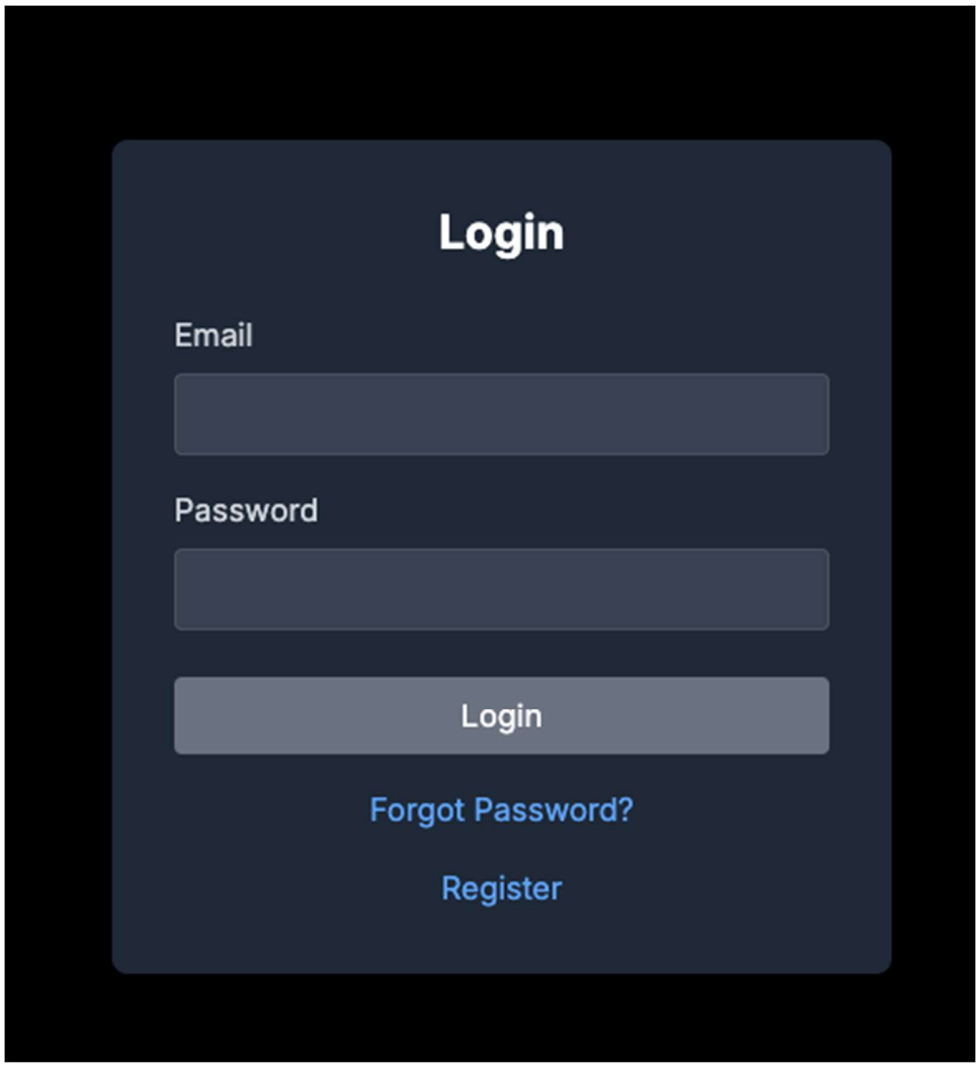


Figure 17- Login page

The authentication process checks user credentials through Supabase's authentication service. Upon successful login, the user's session is confirmed, and their information is synchronized with a specific database table. This synchronization process ensures that user metadata, including role, name, and tax identification number (NIF), is either updated or inserted into the database, with email serving as the unique identifier to prevent duplicates. In the event of errors during the login or synchronization processes, users receive appropriate notifications to guide them.

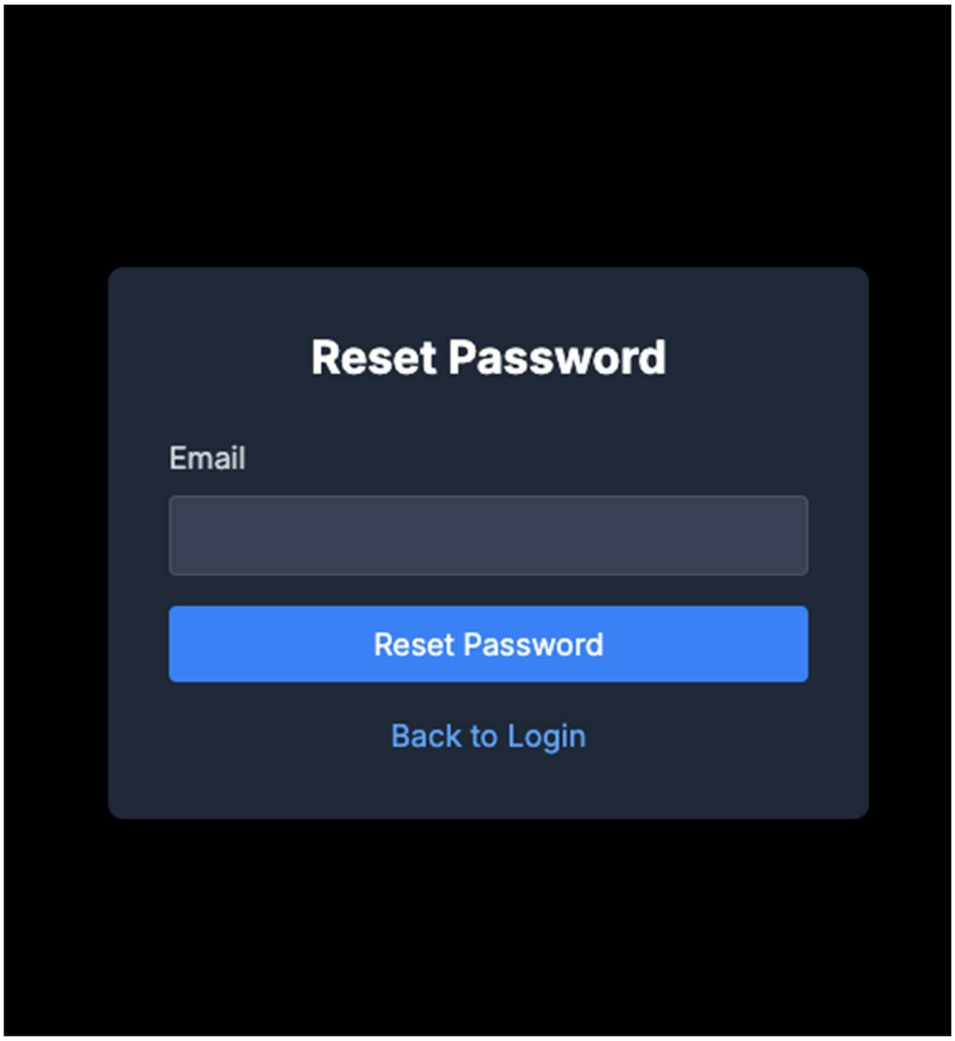


Figure 18 - Reset password page

For those requiring password recovery, the page includes a dedicated mode that facilitates this process. It sends a password reset email via Supabase, which contains a link directing users to a reset page. The interface also offers navigation options between login, password recovery, and account registration, thereby ensuring a seamless experience tailored to various user requirements.

To improve usability, the login button is disabled when mandatory fields are not filled out, thereby preventing invalid submissions. Clear error messages and feedback are provided to

assist users in understanding and resolving any issues encountered during the login process. This login page is meticulously designed to deliver a secure and user-friendly authentication experience, effectively utilizing Supabase's functionalities for both authentication and database management.

4.1.2.2 Register Page

We developed a registration interface that enables users to create an account while ensuring the validation of their inputs and the secure management of user information through Supabase. This interface features a form where users can enter details, including their name, email address, password, password confirmation, tax identification number (NIF), and designated role. The design of the interface aims to facilitate a seamless registration experience, providing clear feedback regarding any issues encountered during the submission of the form.

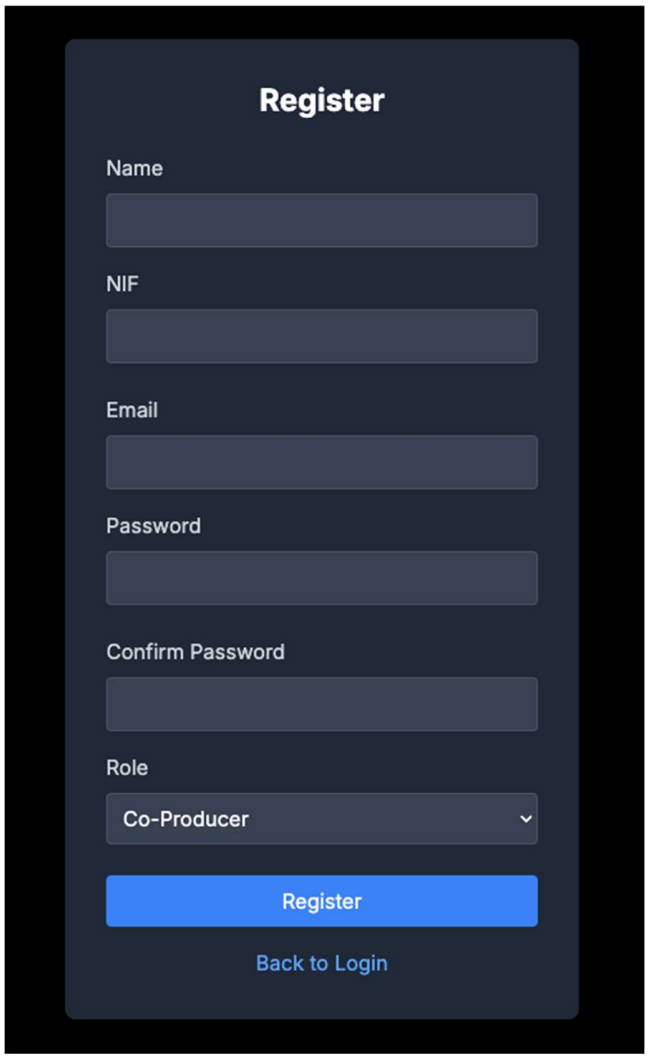


Figure 19 - Register page

The registration procedure integrates input validation mechanisms to confirm that the data provided is both accurate and complete prior to the user's registration attempt. This validation encompasses checks for mandatory fields such as name and email, verification of a valid NIF consisting of exactly nine digits, and assurance that the password adheres to specified length requirements and corresponds with the confirmation input. These validation measures are instrumental in preventing the submission of invalid or incomplete data, with user-friendly error messages presented whenever discrepancies are identified.

Upon successful validation of the form, the registration process advances by securely transmitting the user's information to Supabase via the signUp method. In addition to the email and password, the registration request incorporates supplementary metadata, including the user's role, name, and NIF. If the registration is completed successfully, the user is redirected to the main page of the application. Should any errors arise during the registration process, a clear message is provided to assist users in identifying and rectifying the issue.

Furthermore, the page offers flexibility by allowing users to choose their role from predefined categories such as "Producer" or "Co-Producer." A "Back to Login" button is also included to enhance navigation, enabling users to transition effortlessly between the login and registration processes. In summary, this registration interface prioritizes data integrity and security while delivering a user-friendly experience for account creation.

4.1.2.3 Reset Page

A password reset page has been developed to facilitate the secure updating of user account passwords. This page features an intuitive and accessible interface, allowing users to input their new password and confirm it for accuracy.

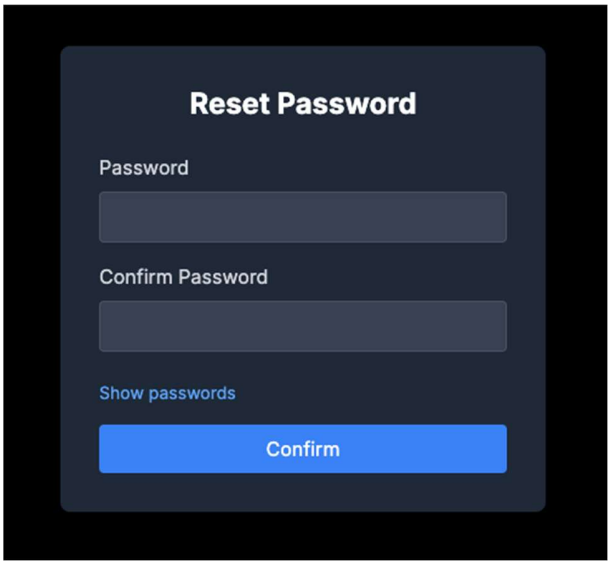


Figure 20 - Reset page

The password reset procedure requires users to enter their new password in two separate fields: one designated for the password and another for confirmation. This dual-entry system is intended to minimize errors by ensuring that users accurately input their desired password twice. In instances where the passwords do not match, users receive immediate feedback, which prevents the submission of incorrect information. Once the passwords are confirmed to match, the new password is securely updated through Supabase's user management API. Following a successful update, users are redirected to the application's main page.

To further improve usability, the page incorporates a toggle feature that allows users to reveal or conceal their passwords while typing. This functionality aids in reducing input errors by enabling users to verify their entries prior to submission. Moreover, the overall design and layout of the page are crafted to ensure that users can navigate the password reset process with ease and clarity.

4.1.2.4 Confirm

A confirmation page has been developed to authenticate user email addresses during the registration process, ensuring that accounts are activated only after successful validation. This page facilitates secure email verification and offers users clear feedback regarding the outcome of the process.

Upon accessing the confirmation page, the system automatically retrieves parameters from the URL, including the verification token, email address, and action type (signup). These parameters are crucial for the email validation process. If any necessary information is absent or incorrect, the page promptly notifies the user that the verification link is invalid.

When the parameters are deemed valid, the page communicates with Supabase to verify the one-time password (OTP) linked to the email. A successful verification results in an update to the user's account status, confirming their registration. Subsequently, the page presents a success message and redirects the user to the login page after a brief delay, ensuring a seamless transition to finalize the signup process. In the event of any verification errors, the user receives a clear message outlining the issue.

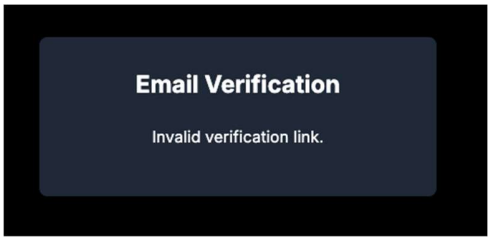


Figure 21 - Email verification

This confirmation page is meticulously designed to provide a secure and user-centric email verification experience. By validating user information and delivering immediate feedback, it significantly enhances the security and dependability of the account creation process.

4.1.3 Private Pages

4.1.3.1 Homepage

A homepage has been developed to function as the primary interface for authenticated users.

Figure 22 - Homepage

The process initiates with a verification of the user's authentication status. In the absence of an active session, a loading message is displayed until the session is confirmed. Upon successful authentication, the system retrieves user-specific data, including their AMAPs. This data is accessed securely through the user's session token, ensuring both privacy and data integrity. Should there be no AMAPs available or if an error arises during the data retrieval, the user receives a relevant notification.

4.1.3.2 AMAP Page

This page is designed to retrieve and present two essential datasets: products and baskets linked to the chosen AMAP. Upon initialization, the page executes two distinct fetch requests—one targeting products and the other focusing on baskets—utilizing secure access tokens derived from the user's session.

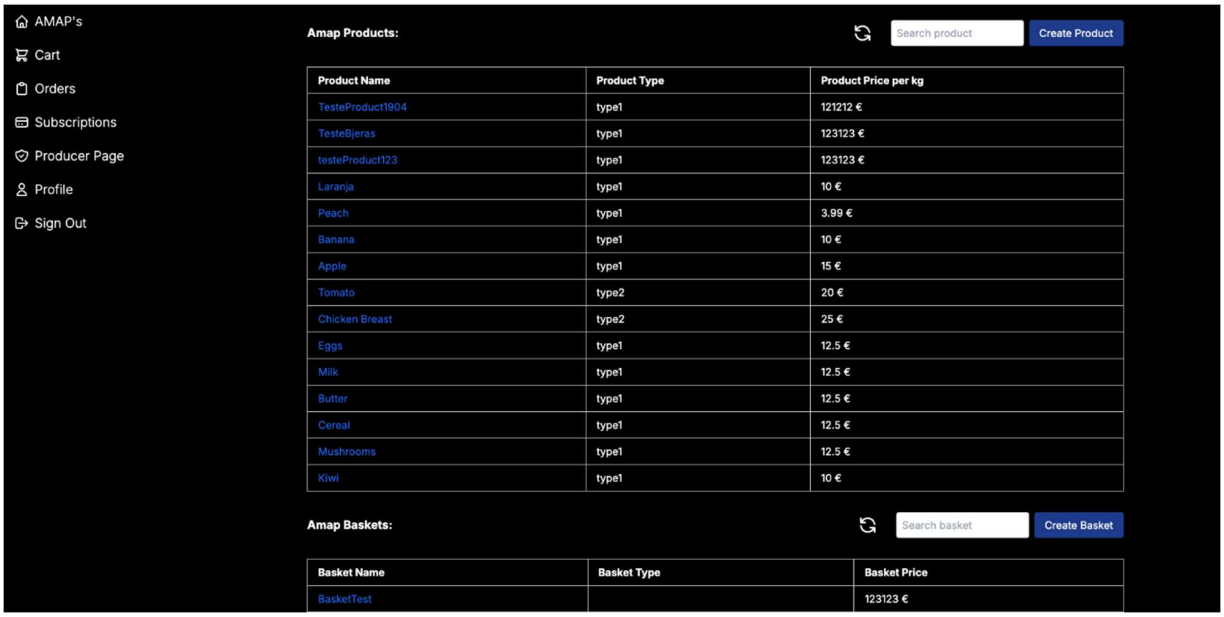


Figure 23 - AMAP page

The retrieved data is subsequently organized into two interactive tables, which facilitate user engagement through exploration, sorting, and filtering of the displayed information. Each table is equipped with real-time filtering capabilities based on user-defined search queries and allows for sorting by various columns, including name, type, or price. This feature set offers users an effective means to access and analyze intricate details, thereby promoting seamless navigation and an improved user experience in managing AMAP-related data.

Additionally, if the user's role permits, they can create new products and baskets directly from the interface, further enhancing data management capabilities for authorized users.

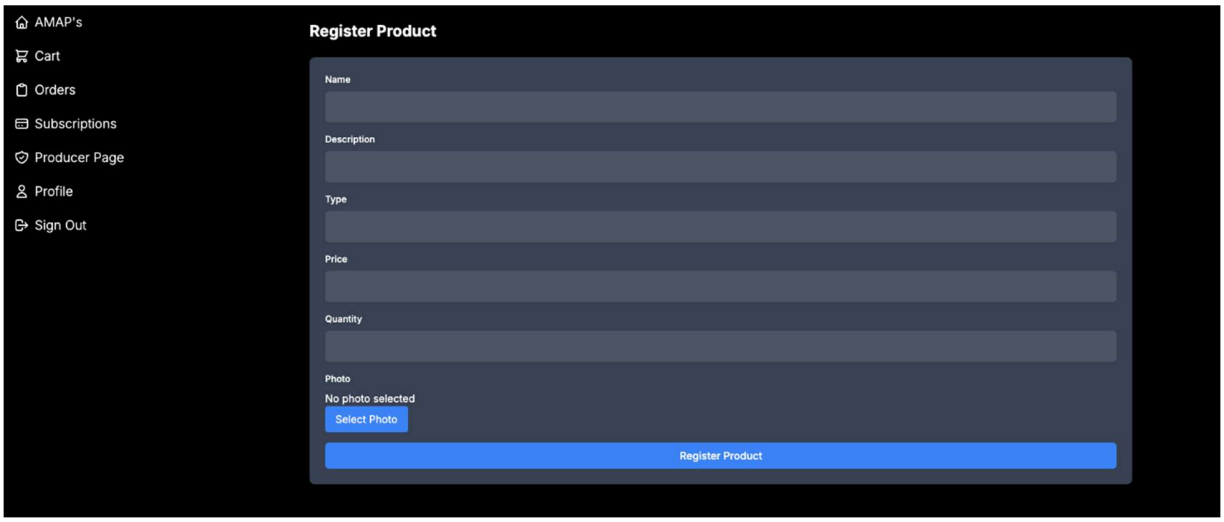


Figure 24 - Register product page

We created a page for product registration that allows authorized users to input product details, select a photo, and submit the data for creation. It includes fields for the product's name, description, type, price, quantity, and photo, with validation to ensure completeness. Users can search and select images through a modal, enhancing flexibility in assigning product photos.

Figure 25 - Create basket page

We also created a page for basket registration that allows authorized users to input basket details, select associated products, and assign a photo before submitting the data for creation. The page retrieves a list of available products and enables users to select multiple items to include in the basket, with real-time data loading to ensure accuracy. Users can provide essential information such as the basket's name, description, price, weight, and associated products, and can select a photo through a searchable image modal. Once all the details are completed, the basket is securely submitted to the backend for creation.

4.1.3.3 Product Page

This product details page provides a view of a selected product, including its name, description, price, available quantity, and associated producer information. Users with the appropriate role, such as "Co-Producer," can subscribe to the product directly, enabling streamlined access.

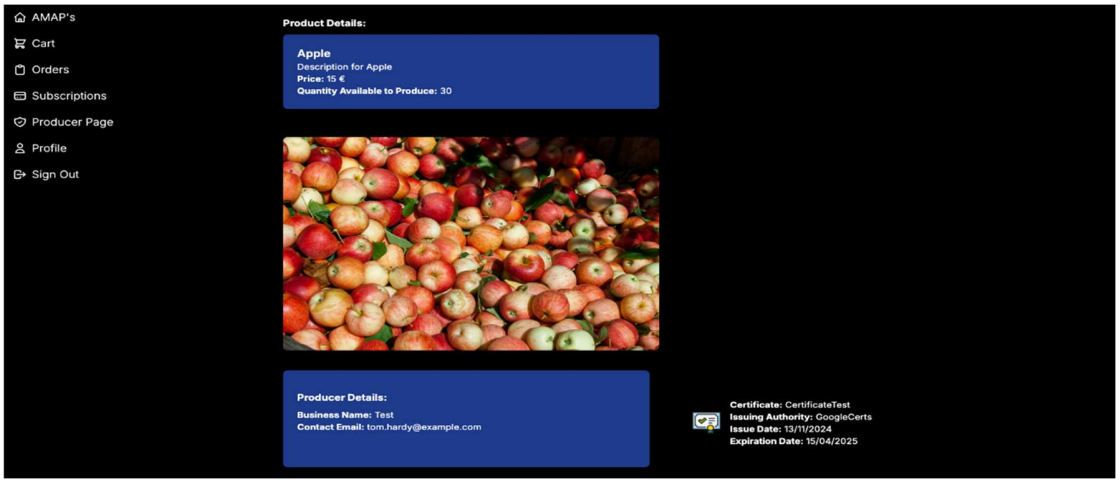


Figure 26 - Product page

The page also showcases a product image and details about certifications issued to the producer, offering additional transparency and credibility. It ensures a seamless user experience by fetching product data dynamically and supporting user-specific actions.

4.1.3.4 Basket Page

This basket details page provides a complete view of a selected basket, including its name, description, price, weight, and associated producer information. Users with the appropriate role, such as "Co-Producer," can subscribe to the basket directly, facilitating streamlined access to its contents.

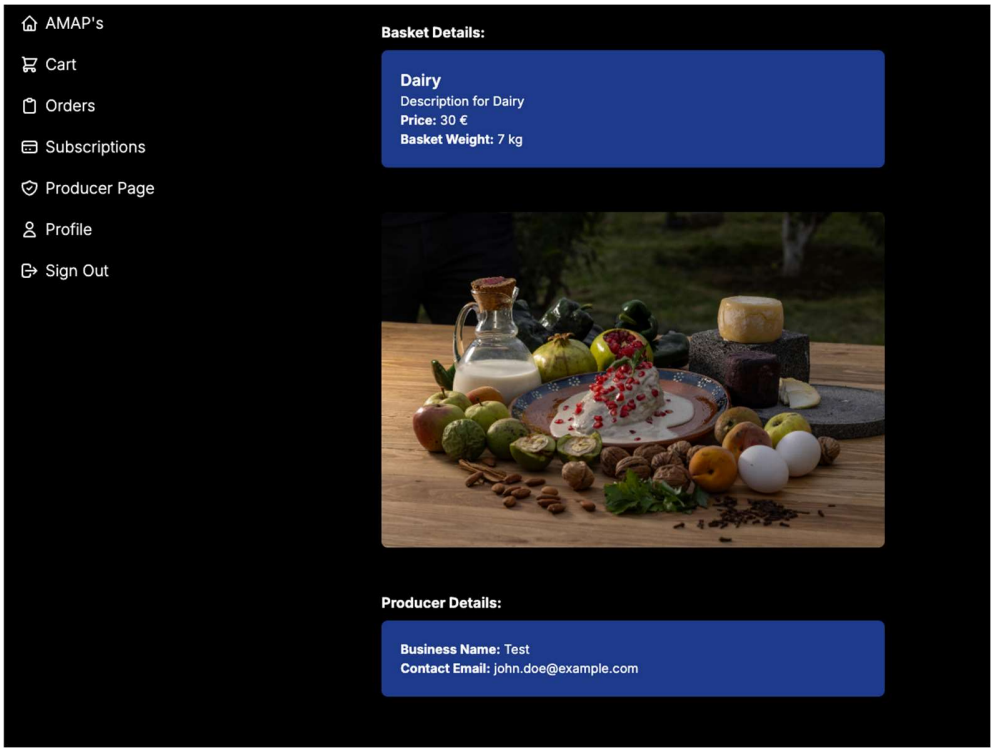


Figure 27 - Basket page

The page also displays a basket image, if available, along with the producer's contact and business details, ensuring transparency and accessibility. It dynamically loads data based on the selected basket and user session, offering a user-friendly and interactive experience.

4.1.3.5 Orders Page

The orders page provides an overview of all user orders, complemented by a KPI summary at the top of the page. The KPI section highlights key metrics such as total orders, total cost, and total amount paid, offering a quick snapshot of overall performance. Below the KPIs, a detailed list of orders is displayed, with each order showcasing its date, status, total cost, and paid amount.

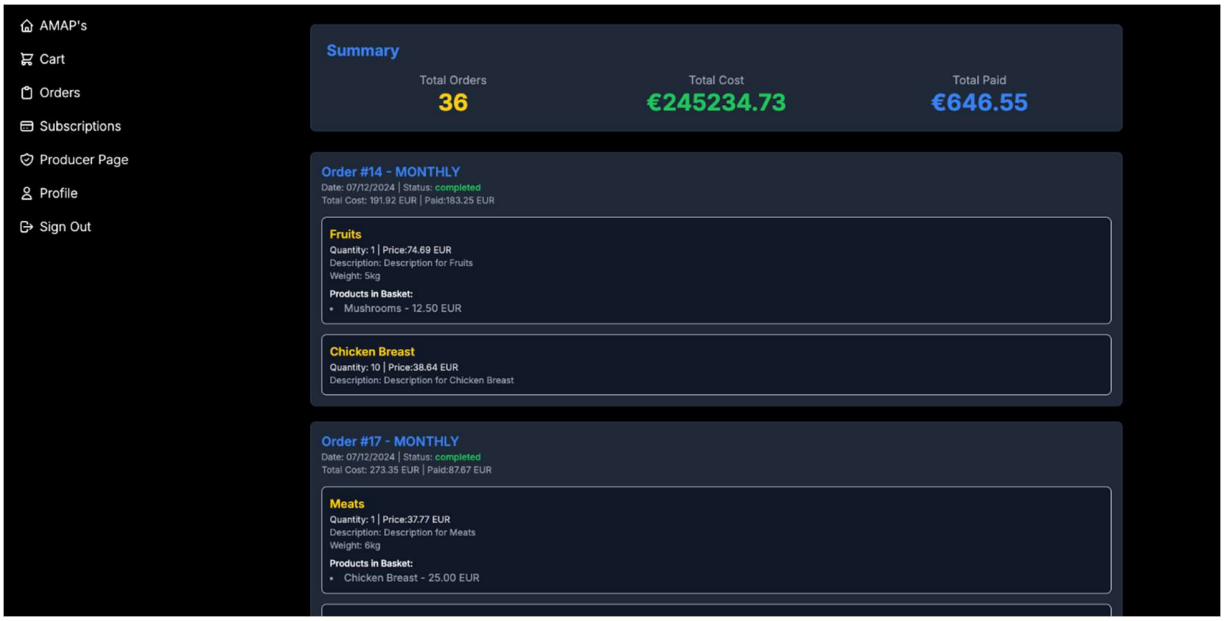


Figure 28 - Orders page

Users can also view the specifics of each order, including details of individual products and baskets, their quantities, prices, and associated descriptions. This layout ensures a clear and comprehensive view of order history and related data.

4.2 Backend

For the backend, a Node.js API was implemented using the Express framework in order to ensure the entire database connection, providing the necessary endpoints for the solution to work.

Middleware was implemented in the API to ensure user authentication, as well as validating their permissions to use certain endpoints. An error handler was also implemented to respond accordingly. As mentioned above, a Bearer token is used to validate access to the API, as well as validating your authentication in the service.

All orders follow code structuring and good software practices, and each route has its own associated controller to manage the order. Each controller for making requests to the database has an associated service for direct actions to the DB.

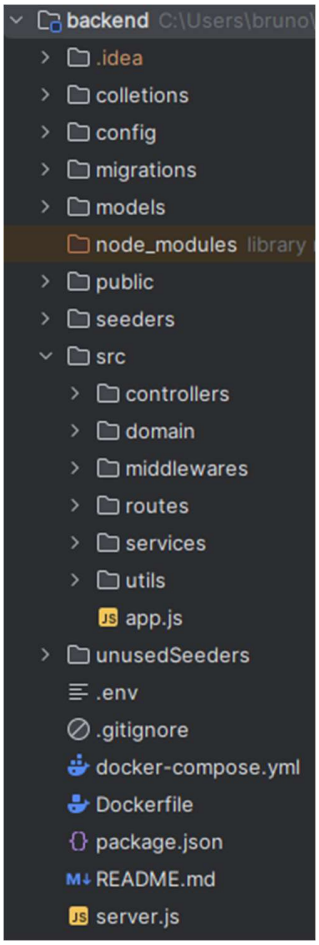


Figure 29 - BE structure

4.2.1 Middleware

The following middleware was implemented in the backend:

Figure 30 - Middleware structure

- **Authentication**: In this middleware, the bearer token is validated in every request if it exists and if it is valid. As well as through the token, we validate that the session is valid and is not expired in the authentication service used, which is Supabase. In this middleware, once validation and authentication is done with Supabase, a user session is also created, improving performance and validations during the API.

- **Error Handler**: It is a middleware used to handle errors in a centralized way, as well as ensuring that the API returns coherent information in accordance with the error detected. It also deals with unexpected errors, always returning an error to the user.

- **Permissions**: In this middleware and according to the user's role, it is validated whether they have permissions for certain endpoints made available. There are some endpoints and/or actions that are limited according to the user's role.

Example:

```
{
    "message": "Invalid or expired token",
    "details": "invalid JWT: unable to parse or verify signature, token has invalid claims: token is expired"
}
```

4.2.2 Controllers / Services

The API was structured to follow good practices and separate the business areas. It is therefore structured according to the routes structure, with each group having its own controller, allowing requests to be managed and processed. Each group also has a service, allowing requests to the database.

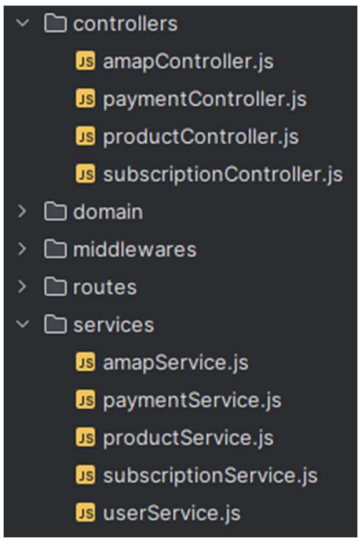


Figure 31 - Controllers/service structure

In the services for DB requests, as already mentioned, Sequelize was used, which will be detailed in the database topic. Sequelize allows queries to be made in a simple way, and was used in all the services.

```
const productDetails = await Product.findAll({
    attributes: ['id', 'name', 'description', 'type', 'price', 'quantity', 'photoUrl'],
    where: {
        id: productID,
    },
    include: [
        {
            model: Producer,
            attributes: ['id', 'businessName'],
            required: true,
            include: [
                {
                    model: User,
                    attributes: ['id', 'email', 'nif'],
                },
                {
                    model: Certificate,
                    attributes: ['id', 'name', 'issuingAuthority', 'issueDate', 'expirationDate'],
                },
            ],
        },
    ],
});
```

4.2.3 Logger

To improve the functioning of the API, a logger system was implemented, allowing requests to be recorded. Another advantage of the logger is that, at this stage of development, it allows rapid analysis of the path and action of each request, facilitating development and possible corrections and improvements.

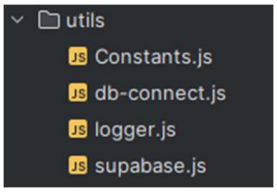


Figure 32 - Utils structure

Example:

```
info: Retrieve CoProducer Payment list
info: Incoming request: GET /payment
info: Fetching data user:
Executing (default): SELECT "email", "role", "AMAPId" FROM "Users" AS "User" WHERE "User"."email" = '1090470@isep.ipp.pt';
info: Retrieved user data: {"email":"1090470@isep.ipp.pt","role":"Co-Producer","AMAPId":4}
info: Request getOrderList
info: Requesting CoProducer Payment list
```

4.2.4 Routes

In order to organize the structure of the routes used, they were grouped according to the basis of the structure and functions of the platform, so the following groups were created:

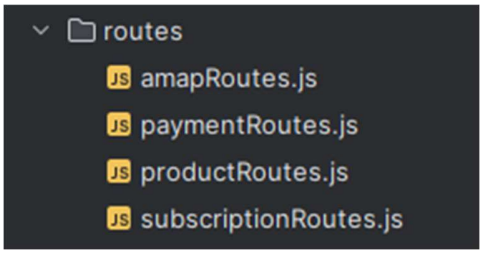


Figure 33 - Routes structure

We are going to detail all the available groups, as well as the endpoints available in each of the groups, but for details of each endpoint and the arguments required, the technical documentation and arguments required for each of them are available via swagger.

4.2.5 AMAP endpoints

In the "AMAP" group, the endpoints related to the actions needed to manage AMAPs have been implemented, and at this stage the following endpoints are available:

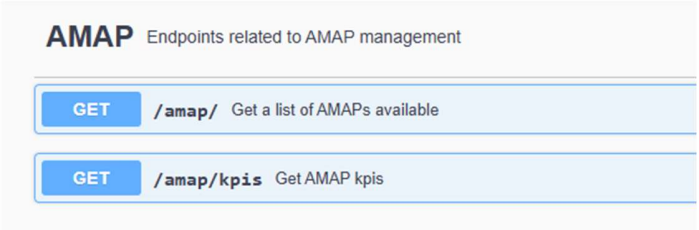


Figure 34 - AMAP endpoints

4.2.5.1 List AMAP's

This endpoint provides a list of available AMAPs, giving simple information such as their name, description and Admin users.

Route: /amap

Type: GET

All users of the application have access to this endpoint, as only basic AMAP information is available.

Response:

4.2.5.2 AMAP KPIS

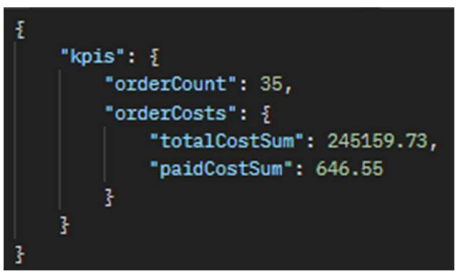
In this endpoint for a given AMAP, based on the orders placed, the number of orders is calculated, the total value of the purchases, and the amount paid at the time.

In terms of permissions, users can only obtain the KPIs of the AMAPs they have access to, and there are no role limitations.

Route: /amap/kpis

Type: GET

Response:



4.2.6 Products endpoints

In the products route, there are endpoints related to product management, such as a list of available products, product details or the possibility of adding new products.

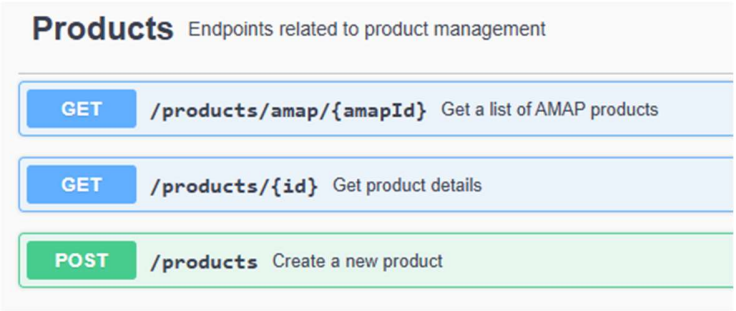


Figure 35 - Products endpoints

4.2.6.1 Products list

This endpoint provides a list of all the products made available by the producers associated with the respective AMAP. The information provided is simple, with only the id, description, type, price and quantity.

Route: /products/amap/{amapId}

Type: GET

In terms of permissions, the endpoint is available to all users who have access to the respective AMAP.

4.2.6.2 Products details

This endpoint provides detailed information on the respective product. It has a section for the product, such as id, name, description, type, price, quantity, photoUrl. A second section for the details of the producer, with its id and bussinessName. A section for the user responsible for the producer, with their id, email and nif. Also, if available, information on the producer's certificate, with the id, name, issuingAuthority, and expiration date.

Route: /products/{productId}

Type: GET

In terms of permissions, the endpoint is available to all users who have access to the respective AMAP associated with the product.

Response:


```
"productDetails": [
    {
        "id": 17,
        "name": "Cereal",
        "description": "Description for Cereal",
        "type": "type1",
        "price": 12.5,
        "quantity": 70,
        "photoUrl": "https://images.pexels.com/photos/3872369/pexels-photo-3872369.jpeg",
        "Producer": {
            "id": 5,
            "businessName": null,
            "User": {
                "id": 14,
                "email": "tom.hardy@example.com",
                "nif": 918726391
            },
            "Certificates": [
                {
                    "id": 1,
                    "name": "Rei dos Frangos",
                    "issuingAuthority": "Luis Filipe Vieira",
                    "issueDate": "2024-11-13T17:01:24.000Z",
                    "expirationDate": "2025-04-15T16:01:33.000Z"
                }
            ]
        }
    }
]
```

4.2.6.3 Create new product

In order to add new products to the platform, an endpoint is provided. This endpoint allows you to create a new product by sending the following mandatory data: name, description, type, price and quantity. The photoUrl field is not mandatory. When the product is created, it is associated with the producer who created it.

Route: /products/

Type: POST

Example arguments:

```
{
    "name": "Kiwi",
    "description": "Kiwi XPTO",
    "type": "type1",
    "price": 10,
    "quantity": 12,
    "photoUrl": "test.jpg"
}
```

In terms of permissions, this endpoint is only available to users with the Producer role. If the users is a coproducer role for example get an error code:

```
{
    "message": "Forbidden: User do not have access to Producer Endpoints."
}
```

Response:

```
"success": true,
"message": "Product created successfully",
"product": {
    "id": 24,
    "name": "Kiwi",
    "description": "Kiwi XPTO",
    "type": "type1",
    "price": 10,
    "quantity": 12,
    "createdAt": "2024-12-08T19:19:50.279Z",
    "updatedAt": "2024-12-08T19:19:50.284Z",
    "producerId": 9,
    "photoUrl": "test.jpg"
}
```

4.2.7 Basket endpoints

Still within the products groups, we have Baskets, being a subgroup of products. Because they are identical, functioning in the same way, only they can have multiple associated products.

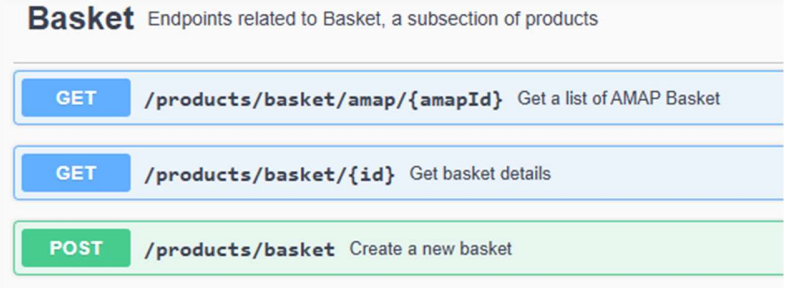


Figure 36 - Basket endpoints

4.2.7.1 Basket list

This endpoint provides a list of all the baskets made available by the producers associated with the respective AMAP. The information provided is simple, with only the id, description, type, price, quantity and the weight.

Route: /products/basket/amap/{amapId}

Type: GET

In terms of permissions, the endpoint is available to all users who have access to the respective AMAP.

Response:


```
"baskets": [
    {
        "id": 9,
        "name": "BasketTest",
        "description": "BasketTest",
        "type": null,
        "photoUrl": "default-photo-url.jpg",
        "price": 123123,
        "weight": 123123123
    },
```

4.2.7.2 Basket details

This endpoint provides detailed information on the respective basket, similar to the product. It has a section for the bascket, such as id, name, description, type, price, weight, photoUrl. A second section for the details of the producer, with its id and bussinessName. A section for the user responsible for the producer, with their id, email and nif. Also, if available, information on the producer's certificate, with the id, name, issuingAuthority, and expiration date. Being a basket, there is a list of products that belong to the basket, which provides information about the respective product, its id, name, description and price.

Route: /products/basket/{basketId}

Type: GET

In terms of permissions, the endpoint is available to all users who have access to the respective AMAP associated with the product.

Response:


```
"basketDetails": [
    {
        "id": 22,
        "name": "Fruit basket",
        "description": "Fresh basket fruit",
        "photoUrl": "test.jpg",
        "price": 15,
        "weight": 123,
        "Producer": {
            "id": 9,
            "businessName": null,
            "User": {
                "id": 12,
                "email": "1090470@isep.ipp.pt",
                "nif": 501501501
            }
        },
        "Products": [
            {
                "id": 10,
                "name": "Banana",
                "description": "Description for Banana",
                "price": 10
            },
            {
                "id": 11,
                "name": "Apple",
                "description": "Description for Apple",
                "price": 15
            }
        ]
    }
]
```

4.2.7.3 Create new basket

In order to add new basket to the platform, an endpoint is provided. This endpoint allows you to create a new basket by sending the following mandatory data: name, description, price and wieght. The photoUrl field is not mandatory. The other mandatory argument is products, being an array with the ids of the products that we want to associate with the basket. When the basket is created, it is associated with the producer who created it.

Route: /products/basket

Type: POST

Example arguments:

```
{
    "name": "Fruit basket",
    "description": "Fresh basket fruit",
    "price": 15,
    "weight": 123,
    "products": [
        {"id": 10},
        {"id": 11}
    ],
    "photoUrl": "test.jpg"
}
```

In terms of permissions, this endpoint is only available to users with the Producer role. If the users is a coproducer role for example get an error code:

```
{
    "message": "Forbidden: User do not have access to Producer Endpoints."
}
```

Response:

```
"success": true,
"message": "Basket created successfully",
"basket": {
    "id": 25,
    "name": "Fruit basket",
    "description": "Fresh basket fruit",
    "price": 15,
    "weight": 123,
    "createdAt": "2024-12-08T19:58:38.515Z",
    "updatedAt": "2024-12-08T19:58:38.518Z",
    "ProducerId": 9,
    "photoUrl": "test.jpg",
    "type": null
}
```

4.2.8 Subscription endpoints

To manage subscriptions, a group is available for listing and creating subscriptions.

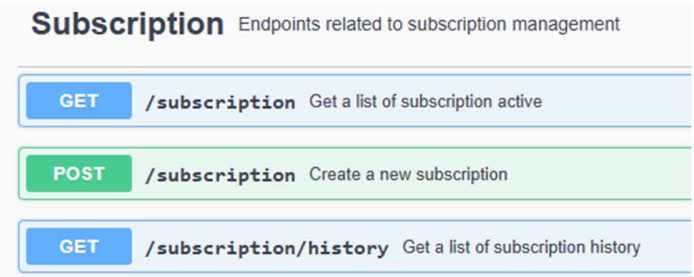


Figure 37 - Subscriptions endpoints

4.2.8.1 Subscription list

To list all active subscriptions, an endpoint has been provided to allow the user to consult all their subscriptions. In the response we have information on the subscription period, its status, and the start and end date. Also information on the order associated with the subscription with information on the products subscribed to.

In this same endpoint, information is made available according to the role, being a coproducer, all of its subscriptions (associated with the user) are listed. If the role is producer, all the subscriptions that have products from the respective producer are listed.

Route: /subscription

Type: GET

In terms of permissions, both roles can access, with the controller managing the information according to each of them.

Response:

4.2.8.2 Subscription history

To list all active subscriptions already completed, an endpoint has been provided to allow the user to consult all their older subscriptions. In the response we have information on the subscription period. Also information on the order associated with the subscription with information on the products subscribed to.

As with the previous endpoint, information is provided depending on the role. All the coproducer's subscriptions, or all the subscriptions with products associated with the producer.

Route: /subscription/history

Type: GET

In terms of permissions, both roles can access, with the controller managing the information according to each of them.

Response:

4.2.8.3 Create subscription

To add new subscriptions, an endpoint has been made available, allowing the user to subscribe to a product or a basket. The periodType is required, which can be weekly or monthly, the type of product (product or basket) is selected and the quantity requested.

The controller calculates the total value of the subscription and the end date of the subscription.

Route: /subscription

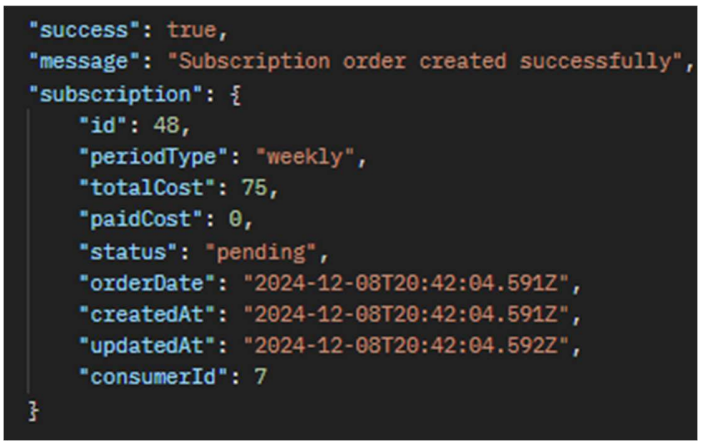
Type: POST

In terms of permissions, this endpoint is only available to co-producers.

Example arguments:

```
{
    "periodType": "weekly",
    "itemType": "product",
    "itemId": 11,
    "quantity": 5
}
```

Response:



4.2.9 Payment endpoints

To manage payments, a group is available for listing and creating payments.

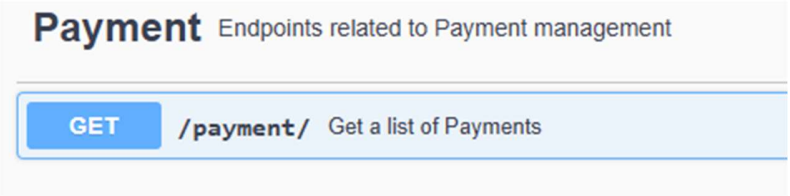


Figure 38 - Payment endpoints

4.2.9.1 Payment list

Endpoint made available to obtain a list of payments made. It provides information on the amount, method, status and order in which the payment was made. This endpoint is also listed according to role, listing coproducer payments, or being a producer, payments for orders related to their products.

The endpoint needs to be upgraded according to the following requirements.

Route: /payment

Type: GET

Response:


```
"payment": [
    {
        "id": 1,
        "amount": 25,
        "currency": "EUR",
        "method": "credit_card",
        "status": "completed",
        "timestamp": "2024-12-08T20:48:45.000Z",
        "paymentProvider": null,
        "orderId": 48,
        "createdAt": "2024-12-08T20:48:53.000Z",
        "updatedAt": "2024-12-08T20:48:57.000Z",
        "userId": 12,
        "User": {
            "id": 12,
            "email": "1090470@isep.ipp.pt",
            "name": "Bruno Santos",
            "role": "Co-Producer",
            "AMAPId": 4
        }
    }
]
```

4.3 Database

To house all the information in the solution and respond to needs, a structure of tables and relations was created and projected in SRS. To implement the database, we used Supabase, creating a PostgreSQL database.

To automatically map our code to the tables, we made use of an ORM like Sequelize.

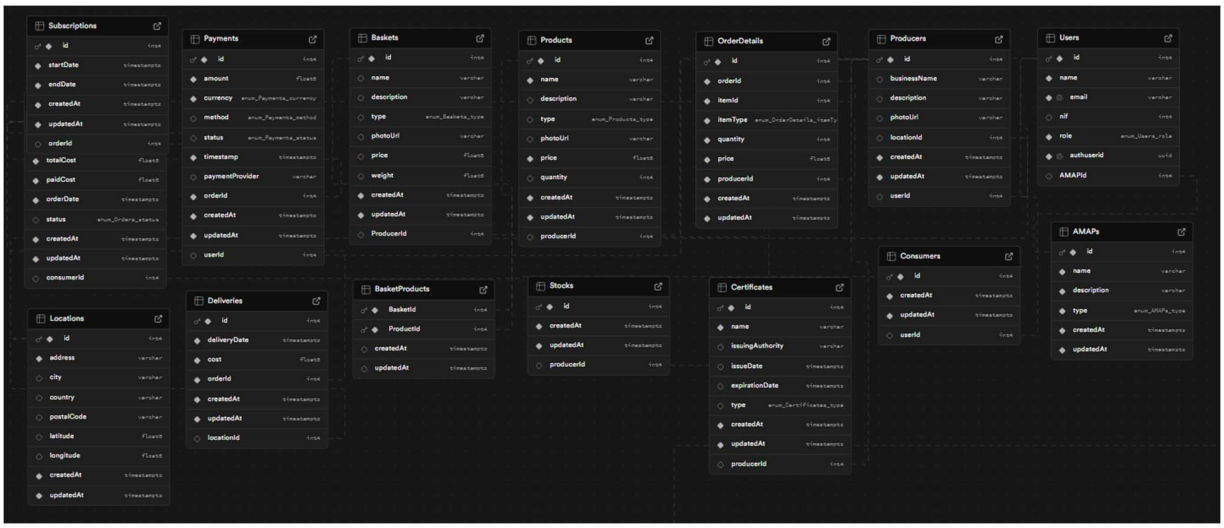


Figure 39 - Database tables

Sequelize is a popular Node.js ORM (Object-Relational Mapping) library that simplifies database interactions by providing a structured way to work with relational databases such as PostgreSQL, MySQL, MariaDB, SQLite, and Microsoft SQL Server. It allows developers to define models for database tables using JavaScript or TypeScript and includes powerful features like model relationships, query building, validation, and migrations. With Sequelize, you can perform CRUD operations (Create, Read, Update, Delete) using a syntax that abstracts SQL queries, making it easier to manage and maintain the database layer of your application.

Sequelize supports advanced ORM functionalities such as associations (e.g., one-to-one, one-to-many, many-to-many relationships), eager loading, transaction management, and schema synchronization. It integrates seamlessly with modern frameworks like Express.js, enabling developers to build scalable applications with ease. By reducing the need for raw SQL, Sequelize allows developers to focus on application logic while ensuring that database interactions are efficient and secure.

Code Examples:

```
const sequelize : Sequelize  = new Sequelize(
    process.env.DB_NAME,
    process.env.DB_USER,
    process.env.DB_PASSWORD,
    {
        host: process.env.DB_HOST,
        port: process.env.DB_PORT,
        dialect: 'postgres'
        // dialectOptions: {
        //     ssl: {
        //         require: true,
        //         rejectUnauthorized: false, // Required for Supabase
        //     },
        // },
    }
);

// Test the connection
sequelize.authenticate()  Guilherme
    .then(() : any | void  => console.log('Database connected...'))
    .catch(err => console.error('Error connecting to the database:', err));
```

This code initializes a Sequelize instance to connect to a PostgreSQL database (supabase) using connection details stored in an environment file (.env)

```
// Sync models
sequelize.sync( options: { force: false }) // Use `force: true` to drop and recreate tables (only for development)
    .then(() : any | void  => console.log('Database synced'))
    .catch(err => console.error('Error syncing the database:', err));
```

This code snippet uses Sequelize to **synchronize the database schema** with the defined models in our application.

```
class User extends Model {}  Show usages  Guilherme
User.init( attributes: {
    name: {type: DataTypes.STRING, allowNull: false},
    email: { type: DataTypes.STRING, allowNull: false, unique: true },
    nif: { type: DataTypes.INTEGER },
    role: { type: DataTypes.ENUM( values: 'Producer', 'Co-Producer', 'Admin', 'AMAP Admin'), allowNull: false },
    // Foreign key to Supabase auth.users
    authuserid: {type: DataTypes.UUID,allowNull: false, unique: true,},
}, options: {
    sequelize,
    timestamps: false,
    modelName: 'User' });
```

This code defines a Sequelize model for a User entity, mapping it to a database table and specifying its structure, validations, and behavior.

```
User.hasOne(Consumer, options: { foreignKey: 'userId' });
User.hasOne(Producer, options: { foreignKey: 'userId' });
User.belongsTo(AMAP, options: { foreignKey: 'AMAPId' });
User.hasMany(Payment, options: { foreignKey: 'userId' });

Stock.belongsTo(Producer, options: { foreignKey: 'producerId' });

Product.belongsTo(Producer, options: { foreignKey: 'producerId' });

Producer.hasMany(Product, options: { foreignKey: 'producerId' });
Producer.belongsTo(User, options: { foreignKey: 'userId' });
Producer.hasOne(Stock, options: { foreignKey: 'producerId' });
Producer.hasMany(Certificate, options: { foreignKey: 'producerId' });

Payment.belongsTo(User, options: { foreignKey: 'userId' });

Order.belongsTo(Consumer, options: { foreignKey: 'consumerId' });
Order.hasMany(Delivery, options: { foreignKey: 'orderId' });

Order.hasMany(Payment, options: {foreignKey:'orderId'});
Payment.belongsTo(Order, options: { foreignKey: 'orderId' });

Producer.belongsTo(Location, options: { foreignKey: 'locationId' });
Location.hasOne(Producer, options: { foreignKey: 'locationId' });

Order.hasOne(Subscriptions, options: { foreignKey: 'orderId' });
Subscriptions.belongsTo(Order, options: { foreignKey: 'orderId' });

Location.hasOne(Delivery, options: { foreignKey: 'locationId' });
```

This code defines and maps the relationships between various Sequelize models, setting up associations such as hasOne, hasMany, belongsTo, and belongsToMany. It effectively specifies how different database tables are connected and how those relationships should be enforced.

4.3.1 Seeders and Migrations

To have data readily available every time we had to do a hard refresh of our database, we made use of sequelize seeders. These seeders will populate each table, insert data that connects using queries and bulkInserts. Although not necessary, we also implemented migrations that specify the tables to be created. These migrations haven't been necessary because everytime we run the project, sequelize is initializing our schema and making sure it is correctly synced with the database.

Code Examples:

```
const consumers = await queryInterface.sequelize.query(
    `SELECT id FROM "Consumers";`,
    {
        type: QueryTypes.SELECT,
    }
);

const currentDate : Date  = new Date();
const orders : any[]  = [];

// Generate orders (single purchase, monthly, weekly)
consumers.forEach((consumer) : void  => {
    // Single purchase order
    orders.push({
        consumerId: consumer.id,
        periodType: periodType.single_purchase,
        totalCost: Math.random() * 100 + 50, // Random total cost between 50 and 150
        paidCost: Math.random() * 50 + 20, // Random paid cost between 20 and 70
        orderDate: currentDate,
        status: status.pending,
        createdAt: currentDate,
        updatedAt: currentDate,
    });

    // Monthly order
    orders.push({
        consumerId: consumer.id,
        periodType: periodType.monthly,
        totalCost: Math.random() * 200 + 100, // Random total cost between 100 and 300
        paidCost: Math.random() * 150 + 50, // Random paid cost between 50 and 200
```

The script generates orders for consumers based on specific types (single purchase, monthly, weekly) and inserts these orders into the Orders table.

The script retrieves non-single-purchase orders and prepares to create corresponding Subscriptions (though this is currently commented out because a trigger is being used to create the subscriptions).

4.3.2 Database Triggers

In our database we also have defined some triggers that will run on insert, in specific tables.

We have 3 triggers in total. The 1st trigger is for Consumer replication, whenever a User with the role “Co-Producer” is inserted, we create a Consumer and reference the userId.

```
CREATE OR REPLACE FUNCTION create_consumer_on_user_insert()
RETURNS TRIGGER AS $$
BEGIN
    -- Check if the role is 'Producer'
    IF NEW.role = 'Co-Producer' THEN
        -- Insert a new Producer record linked to the User's ID
        INSERT INTO public."Consumers" ("userId", "createdAt", "updatedAt")
        VALUES (NEW.id, NOW(), NOW());
    END IF;

    -- Return the newly inserted row
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

The 2nd trigger is for Producer replication, whenever a User with role ”Producer” is inserted, we create a Producer and reference the userId. This ensures coesion.

```
CREATE OR REPLACE FUNCTION create_producer_on_user_insert()
RETURNS TRIGGER AS $$
BEGIN
    -- Check if the role is 'Producer'
    IF NEW.role = 'Producer' THEN
        -- Insert a new Producer record linked to the User's ID
        INSERT INTO public."Producers" ("userId", "createdAt", "updatedAt")
        VALUES (NEW.id, NOW(), NOW());
    END IF;

    -- Return the newly inserted row
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

We also have a trigger to create subscriptions. Given that the order holds all the information necessary, the subscription table just holds very simple data, like a start and end date, and a reference to the order it was triggered from.

```
CREATE OR REPLACE FUNCTION create_subscription_on_order_insert()
RETURNS TRIGGER AS $$
DECLARE
    start_date DATE;
    end_date DATE;
BEGIN
    -- Set the startDate to the current date
    start_date := CURRENT_DATE;

    -- Calculate the endDate based on the periodType
    IF NEW."periodType" = 'weekly' THEN
        end_date := start_date + INTERVAL '7 days';
    ELSIF NEW."periodType" = 'monthly' THEN
        end_date := start_date + INTERVAL '30 days';
    ELSE
        -- Handle other period types or set end_date to NULL for unsupported cases
        end_date := NULL;
    END IF;

    -- Insert a new subscription only if the periodType is not 'single purchase'
    IF NEW."periodType" != 'single purchase' THEN
        INSERT INTO public."Subscriptions" ("orderId", "createdAt", "updatedAt", "startDate", "endDate")
        VALUES (NEW.id, NOW(), NOW(), start_date, end_date);
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

The script above essentially checks to see that the order type is not ”single purchase” meaning that the Consumer chose either a weekly or monthly delivery plan. (these plans can be extended in the future).

If the user chooses a weekly period, then we will create a subscription starting from the current time and ending a week later (7 days). The same applies for Monthly, we add 30 days to the startDate.

Afterwards we insert the subscription in the table.

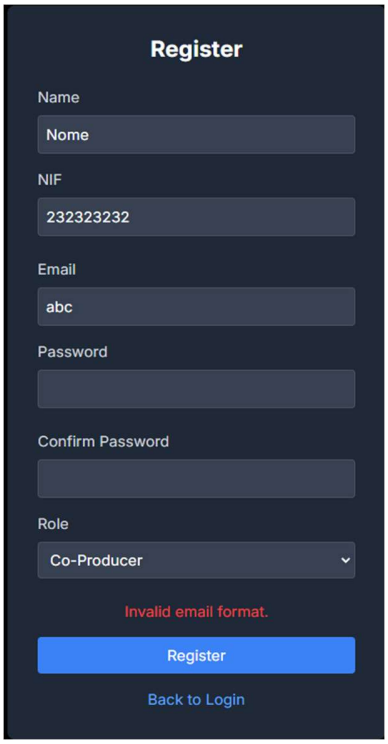
5. Non-functional requirements

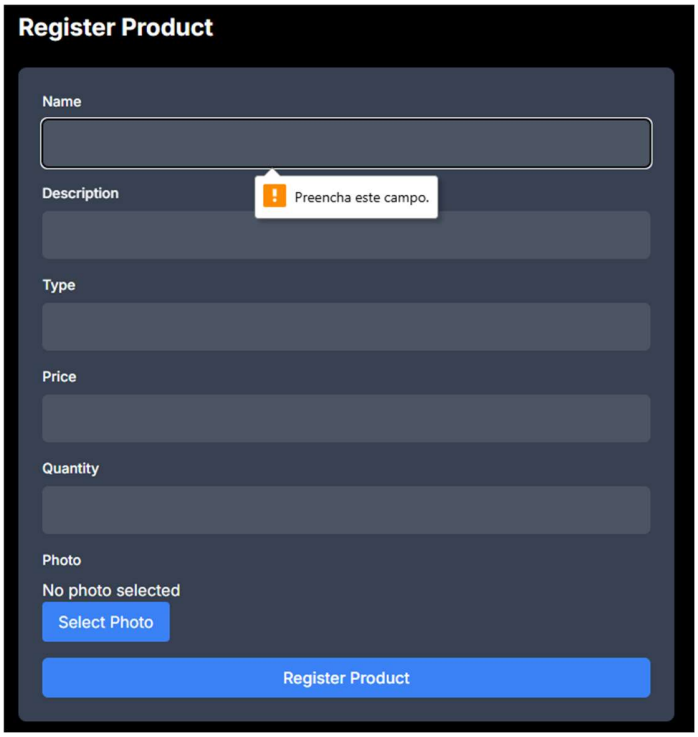
The non-functional requirements implemented in our project align closely with those outlined in the SRS document, with additional feedback incorporated through stakeholder interactions.

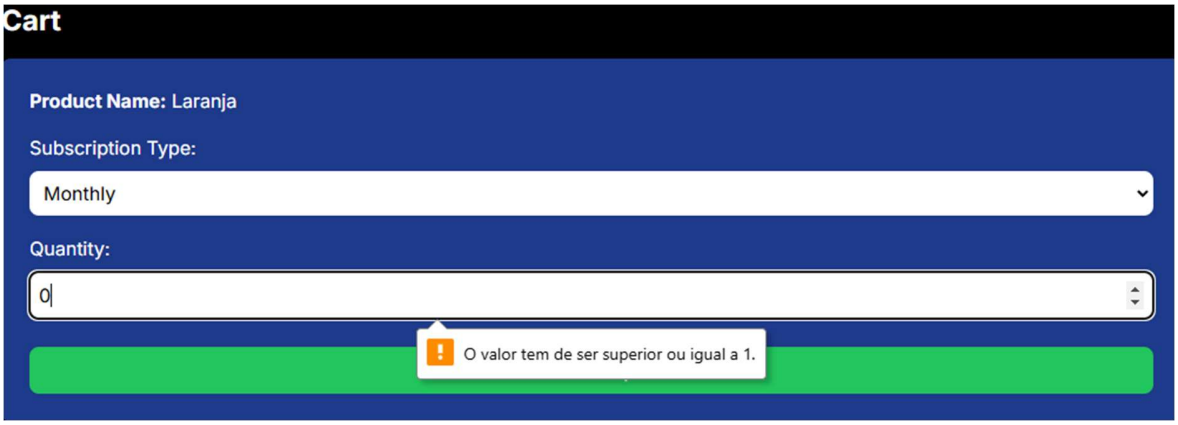
For usability, ARIA attributes were defined to label inputs, buttons, and forms, enhancing accessibility for users with disabilities. Additionally, ARIA attributes like aria-hidden, aria-live, and aria-sort were incorporated to improve dynamic content updates and provide a clearer structure for assistive technologies, ensuring a more inclusive user experience.

```
<input class="p-2 border border-gray-600 rounded bg-gray-700 text-white" aria-label="Enter your Name" type="text" value name="name">
<th class="border border-gray-300 px-4 py-2 text-left" aria-sort="descending">…</th>
<div aria-live="assertive">Subscribe to Product</div>
```

Furthermore, user-friendly error handling is incorporated across various forms, including the registration and product/basket addition processes.

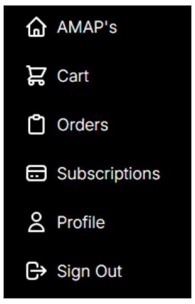




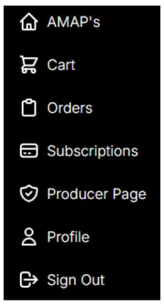


Regarding security, Supabase is the backbone of our authentication system, offering features such as hashed password storage, and role-based access control (RBAC) to restrict user actions based on roles. This approach is mirrored both in the frontend (for controlling access to pages and HTML elements) and in the backend (for securing API endpoints).

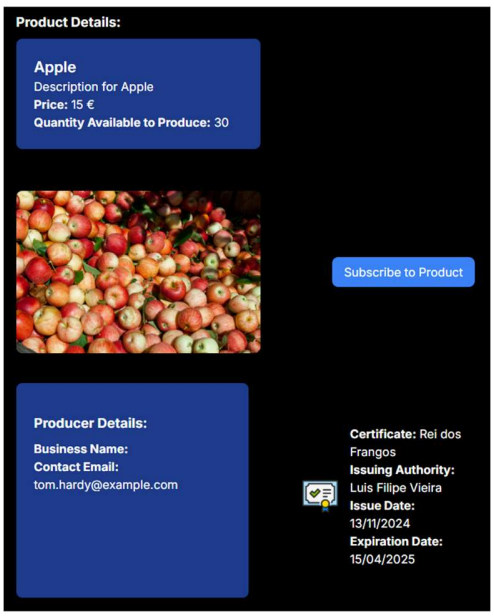
Co-Producer available pages:



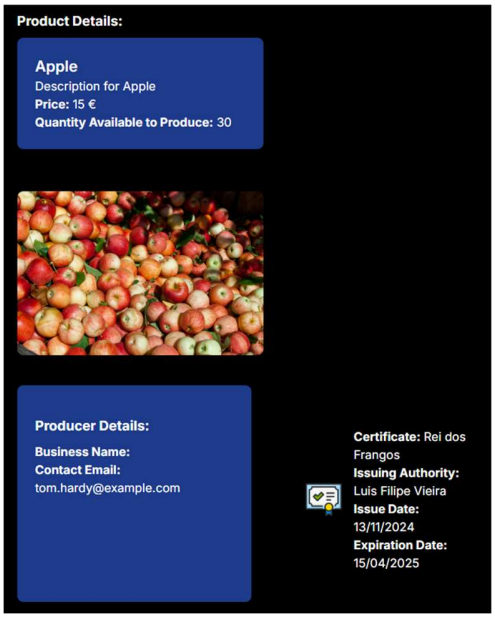
Producer available pages:



Co-Proucer can subscribe to items:



Producer can’t subscribe:



For safety, the implemented strict input validation also helps to prevent malicious attacks such as SQL injection and cross-site scripting (XSS), with specific field restrictions like email formatting and NIF validation.

The application meets compatibility standards by being accessible on the latest versions of Chrome, Firefox, Safari, and Edge, and includes a responsive design that adjusts to various screen sizes, ensuring a consistent user experience across desktop and mobile platforms.

The project also follows RESTful principles for the backend, ensuring that all API endpoints are stateless and adhere to standard HTTP methods. This approach allows the system to be flexible and easily maintainable, ensuring efficient handling of user requests and responses. The RESTful architecture contributes to the application's ability to scale and maintain high performance under varying loads.